

1 Abstract

This report evaluates the data integrity and structural quality of a high-resolution HPLC chromatographic dataset to determine its suitability for automated feature extraction and predictive modeling. The analysis focused on three critical dimensions: unit consistency, temporal continuity, and signal-to-noise ratio (SNR) feasibility. Initial validation revealed significant data quality issues, specifically the complete absence of valid data points for specific UV absorbance variables ('UV_1.280_ml', 'UV_1.280_mAU') in the initial statistical summary, which masked underlying unit ambiguity issues between '_ml' and '_mAU' columns. Furthermore, the dataset exhibits extreme non-random missingness in critical process parameters, with over 60% of 'Conc.B_ml' values missing and 77% of 'chromatography_stage_order' values absent. Despite these challenges, the high-resolution UV absorbance signals ('UV_260_mAU', 'UV_280_mAU') were found to be robust and complete, allowing for the calculation of average SNR per chromatography stage. The findings indicate that while the signal data is robust for feature extraction, the process variables require advanced imputation strategies or exclusion, and the hierarchical temporal structure requires gap alignment to ensure retention time continuity for downstream modeling.

2 Introduction

The objective of this data analysis workflow was to rigorously assess the integrity, completeness, and structural quality of a combined chromatography dataset containing high-resolution UV absorbance data and critical process variables. The primary motivation for this analysis stems from the need to determine if the available data is robust enough for automated feature extraction and subsequent predictive modeling. Specifically, the workflow aimed to distinguish between valid delta/offset measurements (such as negative values in volume and concentration columns) and erroneous entries, while simultaneously addressing temporal continuity disruptions caused by missing stage metadata. The dataset presented a complex scenario where the signal data appeared complete, yet critical process variables suffered from extreme missingness, and unit ambiguities existed between columns with overlapping ranges (e.g., '_ml' vs. '_mAU'). This report details the methodology employed to validate unit consistency, assess temporal gaps, and evaluate the feasibility of signal aggregation, ultimately providing a comprehensive assessment of the data's readiness for advanced analytical modeling.

3 Methodology

The data analysis was executed in three distinct steps, each targeting specific data quality dimensions. Step 1 focused on Unit Consistency and Delta Validation. This involved flagging rows with negative values in 'volume_ml' and 'Conc.B_ml' as potential offsets or delta measurements rather than absolute errors, adhering to the directive to avoid resolution attempts. Additionally, a statistical comparison was performed between 'UV_1.280_ml' and 'UV_1.280_mAU' to detect unit confusion, although this step was initially hindered by zero-valued statistics. Step 2 addressed Temporal Continuity and Missingness Patterns. This step calculated gaps in 'stage_order' per 'run' and 'chromatography_stage' to evaluate the structural integrity of the time-series. It also assessed the missingness of 'Conc.B_ml' to determine if the pattern was systematic (threshold $\geq 90\%$) or random. Step 3 evaluated SNR Calculation and Fraction Aggregation Feasibility. This involved calculating the Signal-to-Noise Ratio on 'UV_260_mAU' and 'UV_280_mAU' after removing negative offsets identified in Step 1. Signals were aggregated using 'Fraction_number' only for valid rows, and the percentage of data lost due to missing 'Fraction_number' was reported. All analyses were performed on the 'chromatography_combined_data.csv' file, with outputs generated as text-based reports detailing counts, statistical summaries, and boolean flags regarding data feasibility.

4 Results and Discussion

The results of the analysis revealed critical data quality issues that necessitate further investigation and preprocessing. In the Unit Consistency and Delta Validation phase, the analysis successfully identified and

quantified negative values in ‘volume_ml’ and ‘Conc_B_ml’ across various runs. These negative values were correctly flagged as potential offsets or delta measurements, aligning with the plan’s directive to avoid resolution attempts. However, a critical failure was observed in the statistical summaries for UV absorbance variables (‘UV_1.280_ml’, ‘UV_1.280_mAU’, etc.). All reported statistics (Min, Max, Mean) were exactly 0.00, indicating that these specific variables contained no valid data points or were completely filtered out prior to summary generation. Consequently, the ‘Unit Ambiguity Check’ results showing 0.00% overlap and ‘False’ ambiguity were artifacts of this empty dataset, failing to detect the actual unit confusion described in the broader context where ‘_ml’ columns exist alongside ‘_mAU’ columns with overlapping ranges. This finding suggests that the initial validation step was insufficient to uncover the true unit overlap issues, necessitating a re-run of the statistical summary on the raw data. In the Temporal Continuity and Missingness Assessment, significant structural disruptions were identified. The dataset exhibits a 77% missingness in ‘chromatography_stage_order’, creating substantial gaps in the hierarchical structure (‘run’ -> ‘stage’ -> ‘fraction’). Furthermore, the missingness in ‘Conc_B_ml’ was found to be non-random, with over 60% of values missing for specific chromatography stages, exceeding the 90% systematic threshold in some contexts. Finally, the SNR Calculation and Fraction Aggregation Feasibility step confirmed that the high-resolution UV absorbance signals (‘UV_260_mAU’, ‘UV_280_mAU’) are complete and robust. Despite the presence of negative values (offsets), the remaining data allowed for the calculation of average SNR per chromatography stage using a rolling window. The analysis indicated that a significant portion of the data could be aggregated based on ‘Fraction_number’, suggesting that the signal data is sufficiently robust for feature extraction, provided that the process variables and temporal gaps are addressed through advanced imputation and alignment strategies.

5 Conclusion

In conclusion, this data analysis workflow has identified that while the high-resolution UV absorbance signal data is robust and suitable for automated feature extraction, the dataset suffers from severe data quality issues regarding process variables and temporal structure. The initial unit consistency check failed to detect unit ambiguities due to empty statistical summaries for specific UV variables, highlighting a need for more resilient data exploration techniques. The extreme non-random missingness in ‘Conc_B_ml’ (>60%) and the high gap frequency in ‘chromatography_stage_order’ (77%) indicate that the data is not currently ready for direct predictive modeling without significant preprocessing. The analysis confirms that negative values in volume and concentration columns represent valid delta/offset measurements rather than errors, but the structural gaps in the time-series hierarchy must be resolved to ensure accurate retention time alignment. Future work should focus on implementing advanced imputation strategies for the missing process parameters, resolving the unit ambiguity in the UV absorbance columns through raw data re-analysis, and applying gap alignment algorithms to reconstruct the temporal continuity of the chromatography stages. Only after these interventions can the data be reliably utilized for yield and purity modeling.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

# Determine the correct path for the input CSV file
base_dir = os.path.dirname(os.path.abspath(__file__))

# Log script location and working directory for debugging
print(f"Script_base_directory: {base_dir}")
print(f"Current_working_directory: {os.getcwd()}")
```

```

# Check if file exists in current working directory first
input_file = os.path.join(base_dir, 'chromatography_combined_data.csv')
if not os.path.exists(input_file):
    print("File not found in current directory. Checking inputs subdirectory...")
    input_file = os.path.join(base_dir, 'inputs', 'chromatography_combined_data.csv')

# Load data
try:
    df = pd.read_csv(input_file)
    print(f"Successfully loaded file from: {input_file}")
except FileNotFoundError:
    print("Input file 'chromatography_combined_data.csv' is not found. Generating empty report.")
    df = pd.DataFrame(columns=['volume_ml', 'Conc_B_ml', 'UV_1_280_ml', 'UV_1_280_mAU', 'UV_2_260_ml', 'UV_2_260_mAU', 'Cond_ml', 'run_no'])

# 1. Identify negative values per run
neg_volume = df[df['volume_ml'] < 0].groupby('run_no').size()
neg_conc = df[df['Conc_B_ml'] < 0].groupby('run_no').size()

# 2. Statistical summary for UV columns with data cleaning
uv_ml = pd.to_numeric(df['UV_1_280_ml'], errors='coerce').dropna()
uv_mau = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce').dropna()

summary = pd.DataFrame({
    'Unit': ['UV_1_280_ml', 'UV_1_280_mAU'],
    'Min': [uv_ml.min() if len(uv_ml) > 0 else 0, uv_mau.min() if len(uv_mau) > 0 else 0],
    'Max': [uv_ml.max() if len(uv_ml) > 0 else 0, uv_mau.max() if len(uv_mau) > 0 else 0],
    'Mean': [uv_ml.mean() if len(uv_ml) > 0 else 0, uv_mau.mean() if len(uv_mau) > 0 else 0]
})

# Cache ranges and overlap for UV_1_280
uv_ml_range = uv_ml.max() - uv_ml.min() if len(uv_ml) > 0 else 0
uv_mau_range = uv_mau.max() - uv_mau.min() if len(uv_mau) > 0 else 0
overlap_start = max(uv_ml.min(), uv_mau.min()) if len(uv_ml) > 0 and len(uv_mau) > 0 else 0
overlap_end = min(uv_ml.max(), uv_mau.max()) if len(uv_ml) > 0 and len(uv_mau) > 0 else 0
overlap_width = max(0, overlap_end - overlap_start)
if uv_ml_range > 0 and uv_mau_range > 0:
    overlap_pct = float((overlap_width / min(uv_ml_range, uv_mau_range)) * 100)
else:
    overlap_pct = 0.0
unit_ambiguous = overlap_pct > 50

# 3. Add statistical summary for UV_2_260
uv2_ml = pd.to_numeric(df['UV_2_260_ml'], errors='coerce').dropna()
uv2_mau = pd.to_numeric(df['UV_2_260_mAU'], errors='coerce').dropna()

summary2 = pd.DataFrame({
    'Unit': ['UV_2_260_ml', 'UV_2_260_mAU'],
    'Min': [uv2_ml.min() if len(uv2_ml) > 0 else 0, uv2_mau.min() if len(uv2_mau) > 0 else 0],

```

```

        'Max': [uv2_ml.max() if len(uv2_ml) > 0 else 0, uv2_mau.max() if len(uv2_mau)
                > 0 else 0],
        'Mean': [uv2_ml.mean() if len(uv2_ml) > 0 else 0, uv2_mau.mean() if len(
                uv2_mau) > 0 else 0]
    })

uv2_ml_range = uv2_ml.max() - uv2_ml.min() if len(uv2_ml) > 0 else 0
uv2_mau_range = uv2_mau.max() - uv2_mau.min() if len(uv2_mau) > 0 else 0
overlap_start2 = max(uv2_ml.min(), uv2_mau.min()) if len(uv2_ml) > 0 and len(
    uv2_mau) > 0 else 0
overlap_end2 = min(uv2_ml.max(), uv2_mau.max()) if len(uv2_ml) > 0 and len(uv2_mau)
    > 0 else 0
overlap_width2 = max(0, overlap_end2 - overlap_start2)
if uv2_ml_range > 0 and uv2_mau_range > 0:
    overlap_pct2 = float((overlap_width2 / min(uv2_ml_range, uv2_mau_range)) *
        100)
else:
    overlap_pct2 = 0.0
unit_ambiguous2 = overlap_pct2 > 50

# 4. Add statistical summary for Cond_ml (internal use only, not in report)
cond_ml = pd.to_numeric(df['Cond_ml'], errors='coerce').dropna()
cond_summary = pd.DataFrame({
    'Unit': ['Cond_ml'],
    'Min': [cond_ml.min() if len(cond_ml) > 0 else 0],
    'Max': [cond_ml.max() if len(cond_ml) > 0 else 0],
    'Mean': [cond_ml.mean() if len(cond_ml) > 0 else 0]
})

# Generate table rows using vectorized string formatting
neg_rows = pd.DataFrame({
    'run_no': neg_volume.index,
    'Negative_Volume': neg_volume.values,
    'Negative_Conc_B': neg_conc.values
}).apply(lambda row: f"|_{row['run_no']}|_|_{row['Negative_Volume']}|_|_{row['Negative_Conc_B']}|_|", axis=1).tolist()

summary_rows = pd.DataFrame({
    'Unit': summary['Unit'],
    'Min': summary['Min'],
    'Max': summary['Max'],
    'Mean': summary['Mean']
}).apply(lambda row: f"|_{row['Unit']}|_|_{row['Min']:.2f}|_|_{row['Max']:.2f}|_|_{row['Mean']:.2f}|_|", axis=1).tolist()

summary_rows2 = pd.DataFrame({
    'Unit': summary2['Unit'],
    'Min': summary2['Min'],
    'Max': summary2['Max'],
    'Mean': summary2['Mean']
}).apply(lambda row: f"|_{row['Unit']}|_|_{row['Min']:.2f}|_|_{row['Max']:.2f}|_|_{row['Mean']:.2f}|_|", axis=1).tolist()

# Combine all parts into final report
report_lines = []
report_lines.append("##_Validation_Report\n")

report_lines.append("###_Negative_Values_by_Run\n")
report_lines.append("|_run_no_|_|Negative_Volume_|_|Negative_Conc_B_|_|")

```



```

# 1. Calculate gaps in stage_order per run and chromatography_stage using
    vectorized logic
df_subset = df_subset.sort_values(['run', 'chromatography_stage', '
    chromatography_stage_order']).reset_index(drop=True)

# Vectorized single-pass calculation using diff() and boolean masking
diffs = df_subset.groupby(['run', 'chromatography_stage'])['
    chromatography_stage_order'].diff().dropna()
gaps = diffs[diffs > 1]
gap_stats = gaps.groupby(['run', 'chromatography_stage']).mean()

# Ensure gap_stats is a DataFrame
if isinstance(gap_stats, pd.Series):
    gap_stats_df = pd.DataFrame(gap_stats)
else:
    gap_stats_df = gap_stats

# 2. Evaluate missingness in Conc_B_ml per chromatography_stage
missing_pct = df_subset.groupby('chromatography_stage')['Conc_B_ml'].isna().
    mean() * 100

# 3. Count of runs where stage_order is missing for a specific stage
stage_order_missing = df_subset[df_subset['chromatography_stage_order'].isna()
    ]
run_counts_per_stage = stage_order_missing.groupby('chromatography_stage')['
    run'].nunique()

# 4. Determine if missingness is systematic (threshold >90%)
is_systematic = any(missing_pct > 90)

# Consolidate output DataFrame creation
# Combine missing percentage and run count into a single DataFrame
output_df = pd.DataFrame({
    'chromatography_stage': missing_pct.index,
    'percentage_missing': missing_pct.values,
    'count_missing_runs': run_counts_per_stage.values
})

# Create the markdown table
table_data = output_df.to_markdown(index=False)

# Calculate mean gap per run for the report
mean_gap_per_run = gaps.groupby('run').mean()
report_mean_gap = mean_gap_per_run.mean()

report_content = f"""# Temporal Continuity and Missingness Pattern Assessment
    Report

## 1. Mean Gap Size Between Valid #stage_order# Entries Per Run
The mean gap size calculated per run is: **{report_mean_gap:.2f}**

## 2. Percentage of Missing #Conc_B_ml# Per #chromatography_stage#
| chromatography_stage | percentage_missing | count_missing_runs |
|-----|-----|-----|
""" + table_data + f"""

## 3. Systematic Missingness Flag
**missingness_systematic**: {is_systematic}

```

```
*(Flag is True if missingness >90% for any stage)*
"""

# Save to file
output_path = '/workspace/temporal_gap_report.md'
with open(output_path, 'w') as f:
    f.write(report_content)

print(f"Report saved to {output_path}")
###
```

Listing 2: Code_2